



**TECNOLÓGICO NACIONAL DE
MÉXICO**

Campus León

Instituto Tecnológico de León

Ingeniería en Sistemas Computacionales

SISTEMAS PROGRAMABLES

**Integración Total MQTT (ESP32 ↔
Python)**

Integrantes:

Varela Ambriz Saul

Rivera Ponce David Eduardo

Ramirez Abundiz Berenice

Fecha: 08 de Mayo 2026

Link del Repositorio en Github:

<https://github.com/SaulVarelaAmbriz/Baston-Inteligente-MQTT.git>

Matriz de Tópicos

Tópico MQTT	Tipo	Dispositivo	Función
bastón/sensores/ultrasónico	Publish	Sensor ultrasónico	Envía la distancia detectada en centímetros
bastón/sensores/pir	Publish	Sensor PIR	Envía el estado de detección de movimiento
bastón/sensores/control	Publish	Control IR	Envía el botón presionado del control remoto
bastón/actuadores/zumbador	Subscribe	Zumbador	Activa el modo localización del bastón
bastón/actuadores/vibración	Subscribe	Motor de vibración	Activa alertas vibratorias
bastón/actuadores/bocina	Subscribe	Bocina PWM	Reproduce alertas sonoras según la distancia

Conclusiones:

David:

Problema:

El reto principal fue que ya tenía la HAL construida de la tarea anterior, pero al intentar conectarla con MQTT, me di cuenta de que algunos sensores, como el receptor IR y el ultrasónico, enviaban datos tan rápido que saturaban el script de Python. Al principio, los mensajes se amontonaban y los actuadores (como el zumbador y el motor de vibración) tardaban varios segundos en responder a los comandos que enviaba desde la computadora.

Solución:

En lugar de rehacer todo, adapté las funciones que ya tenía en mi HAL para que trabajaran en conjunto con el cliente MQTT. Configuré el código para que los sensores solo publicaran información cuando hubiera un cambio real (por ejemplo, cuando el PIR detectara

movimiento) y no todo el tiempo. Esto limpió mucho la comunicación y permitió que la ESP32 pudiera escuchar los comandos de los actuadores sin retrasos, aprovechando la estructura que ya había programado antes.

Conclusión:

Haber hecho la HAL previamente fue una gran ventaja, ya que solo tuve que 'puentear' mis funciones de lectura y escritura con los tópicos de MQTT. Me quedó claro que si el hardware está bien organizado desde el inicio, conectarlo a internet es mucho más sencillo. Logré que todo el sistema funcionara de forma estable y aprendí a gestionar mejor el flujo de datos para no saturar el servicio.

Saul:

Problema:

Durante la integración del protocolo MQTT con la ESP32 surgieron dificultades relacionadas con la comunicación simultánea entre sensores y actuadores. En las primeras pruebas, la publicación constante de datos generaba demasiados mensajes en el broker, provocando retrasos en la recepción de comandos y una respuesta inestable de algunos actuadores. Además, fue necesario adaptar la lógica existente del proyecto para mantener el desacoplamiento entre la comunicación MQTT y el acceso al hardware.

Solución:

Para resolver el problema se aprovechó la arquitectura HAL desarrollada previamente, utilizando sus funciones como intermediarias entre el hardware y el módulo MQTT. Se reorganizó el flujo de telemetría para enviar únicamente eventos relevantes y lecturas importantes, reduciendo la saturación de mensajes. También se separó la lógica de comunicación en un módulo independiente (`mqtt_manager.py`), permitiendo que la ESP32 pudiera administrar correctamente tanto la publicación de sensores como la recepción de comandos remotos sin afectar el funcionamiento general del sistema.

Conclusión:

La implementación de MQTT permitió establecer una comunicación estable entre la ESP32 y el servidor Python, facilitando el monitoreo y control remoto del Bastón Inteligente. La utilización de la HAL simplificó la integración y evitó modificaciones directas sobre el hardware, haciendo el sistema más organizado, escalable y fácil de mantener. Gracias a esta práctica se comprendió la importancia de modularizar el software embebido y de optimizar el flujo de datos en aplicaciones IoT para lograr un comportamiento más eficiente y confiable.

Bere:

Problema:

Uno de los principales inconvenientes durante el desarrollo fue lograr que la comunicación MQTT funcionara correctamente sin afectar el rendimiento de la ESP32. En varias pruebas iniciales, el sistema presentaba comportamientos inestables debido a que algunos sensores realizaban lecturas demasiado frecuentes, ocasionando tráfico innecesario en el broker. Esto provocaba pérdidas de sincronización entre la recepción de datos y la ejecución de comandos

enviados desde el servidor Python, especialmente cuando varios componentes trabajaban al mismo tiempo.

Solución:

Para mejorar el funcionamiento del sistema se reorganizó la estructura del proyecto utilizando una arquitectura modular basada en HAL. De esta manera, la comunicación MQTT quedó separada del acceso directo al hardware, permitiendo que cada componente tuviera una función específica dentro del sistema. También se optimizó la publicación de datos, limitando el envío de información únicamente a eventos importantes o lecturas periódicas controladas, reduciendo considerablemente la carga de mensajes y mejorando la estabilidad de la conexión entre la ESP32 y el broker MQTT.

Conclusión:

La integración total mediante MQTT permitió convertir el Bastón Inteligente en un sistema IoT mucho más flexible y escalable. Gracias a la separación entre la HAL y el módulo de comunicación, fue posible mantener un código más limpio, reutilizable y sencillo de mantener. Además, esta práctica ayudó a comprender la importancia de diseñar arquitecturas desacopladas en proyectos embebidos, ya que facilitan tanto la integración de nuevas tecnologías como la detección y solución de errores durante el desarrollo.